

Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка приложения по варианту ДЭ
и реализация дипломного проекта»

студента _____ IV _____ курса _____ 285 _____ группы

Фамилия _____ Араптанов _____

Имя, отчество _____ Данил Антонович _____

Факультет _____ Инфокоммуникаций, информатики и управления _____

Специальность _____ 09.02.07 «Информационные системы _____

и программирование» _____

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение	5
1 Демонстрационный экзамен	6
1.1 Анализ предметной области и технического задания	6
1.2 Выбор стека технологий и инструментов разработки	7
1.3 Работа с базой данных	8
1.4 Практическая реализация решения	9
1.5 Интерфейс программы	11
1.6 Тестирование программы	13
1.7 Руководство системному программисту	14
1.7.1 Общие сведения о программе	14
1.7.2 Функции программы	14
1.7.3 Минимальный состав технических средств	14
1.7.4 Минимальный состав программных средств	15
1.7.5 Требования к персоналу (системному программисту)	15
1.8 Структура программы	15
1.8.2 Сведения о составных частях программы	16
1.8.2 Сведения о связях между составными частями программы	16
1.8.3 Сведения о связях с другими программами	16
1.9 Настройка программы	17
1.9.1 Настройка на состав технических средств	17
1.9.2 Настройка на состав программных средств	17
1.10 Проверка программы	17
1.10.1 Описание способов проверки	17
1.11 Сообщения системному программисту	18
2 Дипломный проект	19
2.1 Введение дипломного проекта	19
2.2 Анализ предметной области	19
2.3 Требование к своему программному продукту	20
2.4 Обоснование выбора инструментальных средств разработки	22
2.5 Разработка макетов основных форм приложения	26
2.6 Проектирование и разработка базы данных	28
2.7 Разработка пользовательского интерфейса	30
2.8 Доработка приложения	33
2.9.1 Определение доработок	33
2.9.2 Разработка функционала	34

					09.02.07.000022 П.509				
Изм.	Лист	№ докум.	Подпись	Дата					
Разраб.		Араптанов Д.А.			Отчет по МДК 02.02 «Инструментальные средства разработки программного обеспечения»	Лит.	Лист	Листов	
Пров.		Бурумбаев Д.И.					3	42	
Реценз.						УрТИСИ СибГУТИ			
Н. контр.									
Утв.									

Заключение	37
Библиография	38
Приложение А	39
Приложение Б	41

					09.02.07.000022 П. 509	Лист
Изм.	Лист	№ докум.	Подпись	Дата		4

Введение

В текущих условиях цифрового преобразования сферы услуг всё более важную роль играет автоматизация процессов взаимодействия с клиентами и внутреннего взаимодействия персонала. В сервисных центрах нередко применяются устаревшие или не связанные между собой методы регистрации обращений, которые не гарантируют необходимой оперативности обработки запросов и не дают возможности качественно отслеживать деятельность специалистов.

Актуальность данной темы обусловлена потребностью в разработке надёжного программного решения, адаптированного к особенностям функционирования предприятия, занимающегося сервисным обслуживанием и восстановлением климатической техники (кондиционеров, систем вентиляции и обогрева). Внедрение автоматизации способствует росту производительности всего подразделения, значительному уменьшению времени реагирования на поступающие заявки, упрощению контроля над процессом ремонта на всех его стадиях, а также повышению уровня сервиса для клиентов и оптимизации распределения ресурсов.

Целью прохождения учебной практики выступает создание и последующая доработка программного модуля, предназначенного для регистрации и обработки заявок на восстановление климатического оборудования, в контексте подготовки к сдаче демонстрационного экзамена.

Для реализации указанной цели требуется решить следующие задачи:

- изучить техническую документацию и выявить ключевые требования к системе с учётом специфики деятельности сервисного центра;
- выполнить проектирование и создать логически обоснованную базу данных, приведённую к третьей нормальной форме, для надёжного хранения сведений о клиентах, обращениях и персонале;
- разработать пользовательский интерфейс и программную часть для регистрации заявок, корректировки их состояний и распределения обязанностей между исполнителями;
- усовершенствовать систему, добавив возможности для работы новой должности «Менеджер по качеству» с целью быстрого разрешения ситуаций, связанных с затягиванием сроков ремонта;
- подготовить проектную документацию, включающую руководство для системного программиста, в соответствии с действующими нормативными требованиями.

1 Демонстрационный экзамен

1.1 Анализ предметной области и технического задания

В ходе начального этапа выполнения практической работы был осуществлён детальный анализ предметной сферы и пожеланий заказчика. Рассматриваемая предметная область включает процессы технического обслуживания и восстановления климатической техники, в частности кондиционеров, а также вентиляционных и отопительных систем. Ключевая цель создания программного модуля состоит в автоматизации приёма и обработки обращений от клиентов, что позволяет значительно упростить деятельность профильных сотрудников, ускорить реагирование на запросы и более рационально распределять рабочие ресурсы сервисного центра.

Изучение бизнес-процессов выявило, что полный цикл обработки заявки включает несколько основных стадий:

- формирование заявки: клиент передаёт сведения о неисправности, сообщает тип и модель оборудования, описывает характер проблемы и оставляет контактную информацию;
- приём заявки: оператор службы сервиса фиксирует обращение и присваивает ему уникальный номер в учётной системе;
- рассмотрение: на данной стадии выполняется анализ неполадки, устанавливается приоритет задачи и назначается конкретный сотрудник для выполнения работ;
- выполнение: специалист проводит непосредственный ремонт оборудования, при необходимости заказывает требуемые детали либо согласовывает действия с другими работниками;
- отчётность и контроль: после завершения работ исполнитель фиксирует использованные ресурсы, а система предоставляет возможность анализировать обобщённые показатели качества и скорости обслуживания.

Согласно условиям технического задания, разработка осуществляется для заказчика ООО «Конди», а исполнителем выступает сервисный центр «IT-Сфера». Разрабатываемый программный продукт должен обеспечивать реализацию следующих ключевых функций:

- внесение новых обращений и их последующая корректировка;
- контроль текущих состояний заявок, выполнение поиска по номеру;
- ведение статистики деятельности подразделения;
- разграничение прав доступа пользователей на основе авторизации с поддержкой четырёх ролей;
- генерация QR-кода для сбора отзывов о качестве обслуживания;
- возможность создания новых заявок непосредственно заказчиком.

					09.02.07.000022 П. 509	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

1.2 Выбор стека технологий и инструментов разработки

В соответствии с техническим заданием, выбор языка программирования и системы управления базами данных (СУБД) для создания программного модуля остается на усмотрение разработчика. Для реализации полнофункциональной системы учета заявок на ремонт климатического оборудования был выбран современный веб-ориентированный стек технологий: Python, Flask, HTML, JavaScript и SQLite.

Данный выбор обоснован необходимостью создания надежного, безопасного и масштабируемого приложения:

Python и Flask (Backend): использование микрофреймворка Flask на базе языка программирования Python обеспечивает простоту разработки и гибкость настройки. Этот стек позволяет эффективно реализовать сложную бизнес-логику системы учета заявок. Flask является кроссплатформенным решением, что гарантирует стабильную работу программного обеспечения на операционных системах семейства Windows. Встроенные механизмы аутентификации Flask-Login позволяют надежно реализовать авторизацию по логину и паролю, а также разграничить доступ к данным в зависимости от роли пользователя (заказчик, специалист, оператор, менеджер по качеству).

HTML и JavaScript (Frontend): для разработки клиентской части приложения применялась связка языка гипертекстовой разметки HTML и языка программирования JavaScript. HTML формирует структуру веб-страниц, таблиц и форм для ввода данных по заявкам. JavaScript, в свою очередь, обеспечивает интерактивность простого и интуитивно понятного пользовательского интерфейса. Кроме того, использование JS необходимо для реализации функционала форматирования телефонных номеров и подтверждения действий пользователя.

СУБД SQLite и Flask-SQLAlchemy: в качестве базы данных была выбрана легковесная встраиваемая реляционная СУБД SQLite. Она обладает всем необходимым инструментарием для проектирования схемы данных в 3-й нормальной форме (3НФ) с обеспечением строгой ссылочной целостности. Для работы с базой данных использовалась библиотека Flask-SQLAlchemy, предоставляющая удобный ORM-интерфейс для создания таблиц, настройки первичных и внешних ключей, а также импорта начальных данных из предоставленных CSV-файлов.

PyCharm (IDE): основной интегрированной средой разработки выступила PyCharm. Данная IDE предоставляет мощные встроенные инструменты для написания чистого кода, его форматирования и проведения функционального тестирования. Встроенный отладчик позволил оперативно выявлять ошибки, настраивать программную обработку исключительных ситуаций и реализовывать информативные уведомления о действиях пользователя. Поддержка виртуальных окружений и интеграция с системами контроля версий значительно упростили процесс разработки и развертывания приложения.

					09.02.07.000022 П. 509	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

1.3 Работа с базой данных

Для надежного хранения информации была спроектирована реляционная база данных под управлением СУБД SQLite. В соответствии с заданием, главным требованием к разработке ER-модели стало приведение структуры к третьей нормальной форме (3НФ) и обеспечение строгой ссылочной целостности.

На основе анализа технического задания были выделены следующие сущности и их атрибуты:

- users (пользователи): единая таблица для хранения данных всех участников системы, включая клиентов, специалистов, операторов и менеджеров (id – первичный ключ (PK); fio; phone; login; password; type – роль пользователя);

- requests (заявки): центральная таблица учета обращений (id – PK; request_id; number – уникальный номер заявки; start_date; climate_tech_type; climate_tech_model; problem_description; request_status; completion_date; repair_parts; master_id – внешний ключ (FK) к users; client_id – FK к users);

- comments (комментарии): таблица для фиксации дополнительных сведений и информации о заказанных комплектующих (id – PK; comment_id; message; master_id – FK к users; request_id – FK к requests).

Для обеспечения целостности данных связи между сущностями организованы по типу «один ко многим» (1:N) с использованием внешних ключей:

- пользователи (специалисты) и заявки: один специалист может выполнять множество заявок;

- пользователи (клиенты) и заявки: один клиент может оставить множество заявок на ремонт;

- заявки и комментарии: одна заявка может иметь несколько комментариев от специалиста.

Разработанная логическая структура исключает дублирование данных, предотвращает аномалии обновления и обеспечивает высокую скорость выполнения запросов при формировании статистической отчетности.

ERD-диаграмма системы представлена на рисунке 1.1.

Рисунок 1.1 – ERD-диаграмма

Запрос создание базы данных представлен в приложении А.

1.4 Практическая реализация решения

В соответствии с требованиями технического задания, перед непосредственным написанием программного кода был проведен этап проектирования логики работы приложения. Для наглядной визуализации процессов обработки данных были разработаны графические алгоритмы (блок-схемы), выполненные в строгом соответствии с правилами оформления по ГОСТ 19.701.

В рамках проекта были спроектированы две основные блок-схемы:

Основной алгоритм учета заявок: Первая блок-схема описывает базовый алгоритм работы системы учета заявок на ремонт климатического оборудования. Она наглядно демонстрирует общий жизненный цикл обращения: получение входных данных от клиента при регистрации, проверку условий, изменение статусов заявки и формирование выходных данных при завершении ремонта (Рисунок 1.2).

Рисунок 1.2 – Основная блок-схема

Детализированный алгоритм функции расчета: Вторая блок-схема выполняет глубокую детализацию одной из ключевых вычислительных функций модуля – расчета статистики работы отдела обслуживания. Данный алгоритм описывает последовательность циклов и условий, необходимых для подсчета количества выполненных заявок, вычисления среднего времени ремонта и группировки данных для статистики по типам неисправностей (Рисунок 1.3).

Рисунок 1.3 – Детализированная блок-схема

Программное обеспечение реализовано в виде веб-приложения на платформе Flask с использованием архитектурного шаблона, близкого к MVC (Model-View-Controller), что позволяет логически разделить ответственность между компонентами системы.

					09.02.07.000022 П. 509	Лист
						9
Изм.	Лист	№ докум.	Подпись	Дата		

Основные компоненты системы:

- Model (Модели): Сущности базы данных реализованы классами User, Request, Comment в файле models.py. Для привязки классов к таблицам SQLite используются стандартные механизмы SQLAlchemy. Между моделями строго прописаны навигационные свойства, отражающие связи: заявка связана с заказчиком и назначенным специалистом; комментарий привязан к конкретной заявке и специалисту;

- Data (Слой данных): Для работы с базой данных используется объект db из Flask-SQLAlchemy. Он содержит наборы сущностей и конфигурирует связи через определения классов. В соответствии с требованиями к целостности данных, для предотвращения появления «осиротевших» записей установлены соответствующие каскадные ограничения;

- View (Представления): Пользовательский интерфейс реализован с помощью HTML-шаблонов (с использованием Jinja2), расположенных в папке templates. Данные передаются на страницу через контекст рендеринга. На стороне клиента (JS) также реализованы скрипты форматирования телефонных номеров и подтверждения удаления;

- Controller (Контроллеры): Логика обработки HTTP-запросов сосредоточена в файле routes.py, где определены все маршруты приложения и функции их обработки;

Разделение прав доступа и аутентификация: Вход в систему реализован на основе Flask-Login. При успешной проверке логина и пароля пользователь авторизуется, и его данные (ФИО, роль, ID) становятся доступны через current_user. Доступ к маршрутам ограничивается декоратором @login_required, а права детализируются внутри методов на основе значения current_user.type:

- заказчик: может просматривать только свои заявки;
- специалист: может просматривать назначенные ему заявки и отмечать их выполнение;
- оператор/менеджер: имеет доступ ко всем заявкам, может назначать специалистов и удалять заявки;

Работа с заявками (маршруты /requests, /request/<id>, /create_request, /edit_request/<id>):

- список заявок: формируется запрос к базе данных с использованием SQLAlchemy. Реализовано отображение заявок в зависимости от роли пользователя;

- карточка заявки: загружает полную информацию по ID, включая тип климатического оборудования, модель, описание проблемы, данные клиента и специалиста;

- создание заявки: при сохранении автоматически генерируется уникальный номер заявки в формате REQ-XXXX, фиксируется дата создания, а введенные данные проходят валидацию на стороне сервера;

- редактирование: набор доступных для изменения полей определяется в зависимости от роли текущего пользователя;

- удаление: доступно только менеджерам и операторам с обязательным запросом подтверждения;

Модули комментариев и аналитики:

- комментарии к заявкам реализованы через отдельную модель Comment. Специалисты используют этот функционал для фиксации информации о проделанной работе;

- StatisticsController (функция statistics в routes): отвечает за расчет показателей отдела обслуживания. Реализованы SQLAlchemy-запросы для подсчета количества заявок по статусам, вычисления среднего времени выполнения и формирования базовой статистики.

Обработка исключительных ситуаций и интерфейс: Для повышения удобства использования реализована программная обработка ошибок:

- использование конструкции try-except для корректной обработки отсутствующих данных и нарушений прав доступа;

- глобальный перехват исключений при сохранении в БД с выводом понятного сообщения пользователю;

- вывод информативных уведомлений через flash-сообщения об успешном создании заявки, смене статуса или предупреждения о неотвратимых операциях, таких как удаление;

1.5 Интерфейс программы

Пользовательский интерфейс веб-приложения спроектирован интуитивно понятным и простым, что обеспечивает высокий уровень удобства для всех групп пользователей. Все формы, таблицы и компоненты системы выполнены в едином, согласованном визуальном стиле с обязательным наличием заголовков на каждом окне. Для предотвращения неверных действий внедрена система информативных визуальных подсказок и уведомлений, выводящая сообщения об ошибках или успешных операциях. Кроме того, для работы с разными сущностями используются отдельные логические формы, обеспечивающие удобную и последовательную навигацию по приложению.

Интерфейс программы представлен на рисунках 1.4-1.8.

ООО "Конди"

Вход в систему

Логин

Пароль

Войти

Тестовые учетные записи:
Менеджер: login1 / pass1
Специалист: login2 / pass2
Заказчик: login7 / pass7

Рисунок 1.4 – Авторизация

ООО "Конди"Панель управленияЗаявкиСтатистикаВыход

Панель управления

Всего заявок
5

Завершено
1

В работе
2

Новые
2

Список заявок

Номер	Дата	Тип оборудования	Модель	Статус	Действия
REQ-0001	06.06.2023	Кондиционер	TCL TAC-12CHSA/TPG-W	В процессе ремонта	Просмотр
REQ-0002	05.05.2023	Кондиционер	Electrolux EACS/I-09HAT/N3	В процессе ремонта	Просмотр
REQ-0003	07.07.2022	Увлажнитель воздуха	Xiaomi Smart Humidifier 2	Готова к выдаче	Просмотр

Рисунок 1.5 – Главная страница

ФИО клиента

Петров Никита Артёмович

Номер телефона

89219567841

Тип оборудования

Кондиционер

Модель устройства

TCL TAC-12CHSA/TPG-W белый

Описание проблемы

Не охлаждает воздух

Дата создания

06.06.2023

Дата завершения

дд.мм.гггг

Статус заявки

В процессе ремонта

Специалист

Кудрявцева Ева Ивановна

Заказанные комплектующие

Рисунок 1.6 –Добавление новой заявки

Заявка №REQ-0001

Дата создания: 06.06.2023

Клиент: Петров Никита Артёмович

Тип оборудования: Кондиционер

Телефон: 89219567841

Модель: TCL TAC-12CHSA/TPG-W белый

Статус: В процессе ремонта

Описание проблемы

Не охлаждает воздух

Специалист: Кудрявцева Ева Ивановна

Дата завершения: 03.03.2026

Действия

Редактировать

Удалить

Назад к списку

В дашборд

Рисунок 1.7 – Добавленная запись в таблице

Список заявок					
Номер	Дата	Тип оборудования	Модель	Статус	Действия
REQ-0001	06.06.2023	Кондиционер	TCL TAC-12CHSA/TPG-W	В процессе ремонта	Просмотр
REQ-0002	05.05.2023	Кондиционер	Electrolux EACS/I-09HAT/N3	В процессе ремонта	Просмотр
REQ-0003	07.07.2022	Увлажнитель воздуха	Xiaomi Smart Humidifier 2	Готова к выдаче	Просмотр

Рисунок 1.8 – Удаление записи в таблице/ Обновленная таблица после удаление записи

1.6 Тестирование программы

После реализации программы она была протестирована, тесты и их результаты представлены в таблице 1.1.

Таблица 1.1 – Тестирование программы

№	Проверяемая функция	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	Добавление заявки	Корректные данные (тип, модель, проблема, ФИО, телефон)	Запись сохранена в БД, добавлена в список	Сохранено, отображается в списке	Успешно
2	Валидация формы	Пустое поле «Описание проблемы»	Вывод сообщения об ошибке	Выведено сообщение об ошибке	Успешно
3	Редактирование	Изменение статуса на «в процессе ремонта»	Статус обновлен в БД и интерфейсе	Статус успешно обновлен	Успешно
4	Удаление заявки	Нажатие кнопки «Удалить»	Появление запроса на	Окно подтверждения	Успешно

09.02.07.000022 П. 509

Лист
13

Изм.

Лист

№ докум.

Подпись

Дата

№	Проверяемая функция	Входные данные	Ожидаемый результат	Фактический результат	Статус
			подтверждение	отображается	
5	Расчет статистики	Переход в раздел (при 3 выполненных заявках в БД)	Отображение корректных подсчетов	Подсчет совпадает с данными БД	Успешно

По результатам тестирования, все тестирование прошло успешно.

1.7 Руководство системному программисту

1.7.1 Общие сведения о программе

Программа «Учёт заявок на ремонт климатического оборудования» предназначена для регистрации, хранения и обработки заявок на ремонт климатического оборудования. Система обеспечивает централизованный учёт заявок, распределение задач между специалистами и контроль сроков выполнения работ.

1.7.2 Функции программы

Основные функции программы:

- регистрация новых заявок (заказчиком и оператором);
- хранение информации о типе и модели оборудования, описании неисправности, заказанных комплектующих;
- назначение ответственного специалиста по заявке;
- изменение статуса заявки (Новая заявка, В процессе ремонта, Готова к выдаче);
- ведение комментариев по заявке (специалистами);
- просмотр списка заявок с фильтрацией по роли пользователя;
- формирование статистики (количество заявок по статусам);
- аутентификация пользователей по логину/паролю и разграничение прав по ролям (Заказчик, Специалист, Оператор, Менеджер).

1.7.3 Минимальный состав технических средств

Минимальная конфигурация для эксплуатации:

- персональный компьютер с архитектурой x64;

- процессор не ниже 2-ядерного, 2,0 ГГц и выше;
- оперативная память – не менее 2 ГБ (рекомендуется 4 ГБ);
- свободное место на диске – не менее 500 МБ для системы и БД;
- сетевое подключение (для доступа пользователей через браузер);
- монитор с разрешением не ниже 1366×768.

1.7.4 Минимальный состав программных средств

- операционная система: Windows 10/11 или любой дистрибутив Linux;
- установленный Python версии 3.8 и выше;
- установленные пакеты Python из файла requirements.txt;
- браузер пользователя: Microsoft Edge, Google Chrome, Mozilla Firefox (актуальные версии).

1.7.5 Требования к персоналу (системному программисту)

Системный программист, сопровождающий программу, должен:

- знать основы администрирования Windows;
- разбираться в работе Flask-приложений (структура проекта, запуск, логирование);
- уметь работать с SQLite (создание/изменение таблиц, резервное копирование);
- разбираться в механизмах аутентификации Flask-Login;
- уметь читать и анализировать журналы ошибок (лог-файлы приложения).

1.8 Структура программы

1.8.1 Сведения о структуре программы

Программа реализована как веб-приложение на Flask и состоит из следующих основных компонентов:

- точка входа и конфигурация приложения (app/___init__.py);
- слой доступа к данным (Flask-SQLAlchemy и классы моделей в models.py);
- контроллеры (логика обработки запросов в routes.py);
- представления (HTML-шаблоны в папке templates) для отображения интерфейса пользователю;
- статические ресурсы (CSS, JS в папке static).

					09.02.07.000022 П. 509	Лист
						15
Изм.	Лист	№ докум.	Подпись	Дата		

1.8.2 Сведения о составных частях программы

Основные составные части:

- точка входа: `run.py` – запускает приложение;
 - конфигурация приложения: `app/__init__.py` – создание экземпляра Flask, настройка подключения к БД, инициализация расширений;
 - модели данных: `app/models.py` – классы `User`, `Request`, `Comment`, описывающие структуру таблиц и связи между ними;
 - маршруты: `app/routes.py` – все маршруты приложения и функции их обработки с логикой авторизации и работы с данными;
 - утилиты: `app/utils.py` – вспомогательные функции (генерация QR-кодов);
- представления (HTML-шаблоны):
- `base.html` – базовый шаблон с общей структурой страницы;
 - `index.html` – главная страница;
 - `login.html` – форма входа;
 - `dashboard.html` – панель управления;
 - `requests.html` – список всех заявок;
 - `request_detail.html` – детальная информация о заявке;
 - `create_request.html` – форма создания заявки;
 - `edit_request.html` – форма редактирования заявки;
 - `statistics.html` – страница статистики

1.8.2 Сведения о связях между составными частями программы

`run.py` импортирует и запускает приложение из `app/__init__.py`.

`app/__init__.py` создает экземпляры `db` и `login_manager`, которые используются во всем приложении.

`models.py` использует объект `db` для определения классов моделей.

`routes.py` использует `app`, `db`, модели и `login_manager` для обработки запросов.

Шаблоны (`.html`) получают данные из маршрутов и отображают их пользователю.

1.8.3 Сведения о связях с другими программами

Программа не требует подключения к внешним сервисам и полностью автономна. База данных SQLite хранится в файле `instance/repair_system.db` и не требует отдельного сервера БД.

					09.02.07.000022 П. 509	Лист
						16
Изм.	Лист	№ докум.	Подпись	Дата		

1.9 Настройка программы

1.9.1 Настройка на состав технических средств

Обеспечить наличие необходимого объёма ОЗУ и свободного места на диске. Особых требований к настройке оборудования не требуется.

1.9.2 Настройка на состав программных средств

Для корректной работы программы необходимо установить все зависимости из файла requirements.txt:

```
pip install -r requirements.txt
```

После установки зависимостей необходимо выполнить импорт начальных данных:

```
python data/import_script.py
```

Запуск приложения осуществляется командой:

```
python run.py
```

1.10 Проверка программы

1.10.1 Описание способов проверки

Основные способы проверки корректности работы:

1) Функциональное тестирование:

- проверка сценариев входа/выхода и разграничения прав по ролям;
- проверка CRUD-операций по заявкам (создание, просмотр, изменение, удаление);
- проверка поиска и фильтрации заявок;
- проверка формирования статистики.

2) Тестирование интеграции с БД:

- проверка подключения к SQLite;
- проверка корректности работы с первичными/внешними ключами и ограничениями;
- проверка импорта данных из CSV-файлов.

3) Тестирование отказоустойчивости:

					09.02.07.000022 П. 509	Лист
						17
Изм.	Лист	№ докум.	Подпись	Дата		

- поведение при недоступной БД (корректное сообщение об ошибке, отсутствие падения приложения);
- корректная обработка ошибок сохранения.

По результатам испытаний заполняется отдельный документ «Протокол тестирования» с перечнем тест-кейсов, фактическими результатами и статусом.

1.11 Сообщения системному программисту

В процессе работы возможны следующие типичные сообщения и ситуации (таблица 1.2):

Таблица 1.2 – Сообщение, причины и решение

Сообщение / ситуация	Возможная причина	Действия системного программиста
sqlite3.Operational Error: no such table	База данных не инициализирована	Запустить python data/import_script.py для создания таблиц
ImportError: No module named 'flask'	Не установлены зависимости	Выполнить pip install -r requirements.txt
Ошибка авторизации, сообщение «Неверный логин или пароль»	Неверно введен пароль или логин; данные в users некорректны	Проверить запись пользователя в таблице users через debug_db.py
Отказ в доступе при входе на страницу	Пользователь не авторизован или не имеет нужной роли	Проверить, что маршрут защищен декоратором @login_required
Пустой список заявок	В БД нет заявок или неправильно работает фильтрация	Проверить содержимое таблицы requests через debug_db.py

2 Дипломный проект

2.1 Введение дипломного проекта

В современных условиях цифровизации юридической деятельности особое значение приобретает автоматизация процессов документооборота и управления контрагентами. На многих юридических предприятиях используется программное обеспечение для ведения учёта, однако существующие решения часто не соответствуют конкретным требованиям юридической практики, имеют ограниченный функционал или неудобный интерфейс.

Актуальность темы заключается в необходимости создания нового программного продукта, который будет учитывать специфику работы юридических компаний и обеспечит удобный, точный и эффективный учёт документооборота. Автоматизация позволит повысить достоверность данных, ускорить обработку документации, упростить контроль движения документов и сроков, а также снизить вероятность ошибок, возникающих при ручном ведении учёта.

Целью дипломной работы является расширение функционала собственного программного продукта, созданного при написании курсового проекта.

Для достижения данной цели необходимо выполнить следующие задачи:

- определить возможное расширение функционала;
 - описать суть нового функционала и его необходимость;
 - добавить функцию полуавтоматической загрузки файлов;
 - добавить защиту от взлома для авторизации пользователя;
 - добавить систему локализации;
- оформить пояснительную записку.

2.2 Анализ предметной области

Индивидуальный предприниматель Араптанов А.Ю. осуществляет деятельность в сфере оказания юридических услуг. Компания ориентирована на предоставление квалифицированной правовой поддержки частным лицам и малым предприятиям. Юридическая практика включает такие направления, как консультирование, составление договоров и исковых заявлений, представительство интересов в судах и государственных органах.

Предприятие имеет распределенную структуру с двумя офисами, расположенными в г. Екатеринбург (бизнес-центр «Консул») и г. Первоуральск. Наличие двух филиалов обусловлено стремлением расширить клиентскую базу и быть географически ближе к клиентам в разных городах.

В настоящее время ключевые бизнес-процессы предприятия организованы по традиционной, "бумажной" модели. Несмотря на наличие персональных компьютеров и оргтехники (принтеры, сканеры), которые используются

					09.02.07.000022 П. 509	Лист
						19
Изм.	Лист	№ докум.	Подпись	Дата		

преимущественно для подготовки документов и их распечатки, основным носителем информации и инструментом по-прежнему выступает бумажный документ.

Это приводит к ряду системных проблем:

низкая оперативность работы, поиск, обработка и согласование бумажных документов (договоров, исковых заявлений, запросов) между офисами занимает значительное время. Необходимость физической передачи документов из одного города в другой создаёт серьёзные задержки;

высокие транзакционные издержки - расходы на печать, копирование, архивирование и транспортировку бумажных носителей существенно снижают рентабельность предприятия;

риски потери и повреждения информации. Бумажные архивы уязвимы для физического износа, потери или повреждения, что может привести к утрате критически важной информации по клиентским делам;

неэффективное управление задачами и клиентами. Отсутствие единой электронной системы приводит к тому, что учёт рабочих задач, контроль сроков исполнения поручений и ведение истории взаимодействий с клиентами ведётся фрагментарно, зачастую в разных блокнотах или файлах, что затрудняет оперативный контроль и анализ;

сложности масштабирования. Рост клиентской базы и увеличение объёма документооборота при существующей системе неминуемо приведёт к организационному хаосу и потребует найма дополнительного административного персонала для рутинной работы с бумагами.

Таким образом, ключевой проблемой ИП Араптанов А.Ю. является неэффективная, затратная и медленная организация внутренних процессов, основанная на бумажном документообороте. Это сдерживает развитие бизнеса, снижает его конкурентоспособность и повышает операционные риски.

Настоятельно требуется цифровая трансформация — переход на электронный документооборот и внедрение специализированного программного обеспечения, которое автоматизирует основные бизнес-процессы. Такое решение позволит централизовать управление документами, задачами и клиентской базой, обеспечив тем самым повышение скорости, прозрачности и общей эффективности работы предприятия.

2.3 Требование к своему программному продукту

Функциональные требования

Приложение обеспечивает комплексную автоматизацию рабочих процессов с следующими ключевыми функциями:

- управление документами: создание, хранение и организация документов с использованием интеллектуальных шаблонов;

					09.02.07.000022 П. 509	Лист
						20
Изм.	Лист	№ докум.	Подпись	Дата		

- анализ контрагентов: проверка надёжности партнёров через интеграцию с государственными реестрами;
- контроль сроков: автоматическое отслеживание процессуальных сроков с системой уведомлений;
- поиск и аналитика: полнотекстовый поиск и формирование отчетов по документам и контрагентам.

Нефункциональные требования

Производительность и надёжность:

- время отклика интерфейса: не более 1-2 секунд;
- время генерации документов: не более 5 секунд;
- стабильная работа при временном отсутствии интернет-соединения.

Безопасность и удобство использования:

- локальное хранение конфиденциальных данных;
- интуитивно понятный интерфейс на русском языке;
- целостность данных при некорректном завершении работы;
- обучение работе с системой не более 1 часа.

Эксплуатационные требования

Разрабатываемое приложение представляет собой кроссплатформенное десктопное решение с архитектурой, поддерживающей два режима работы: однопользовательский режим с локальным хранением данных и файл-серверный вариант для коллективного использования. Данная архитектура обеспечивает гибкость развертывания и позволяет адаптировать программный продукт под различные масштабы предприятий и организационные структуры.

Техническая реализация:

- язык программирования C#;
- платформа .NET 8;
- фреймворк AvaloniaUI;
- система управления базами данных SQLite.

системные требования:

- процессор от 1 ГГц;
- оперативная память от 4 ГБ;
- дисковое пространство от 500 МБ.

Программное обеспечение:

- Поддерживаемые операционные системы включают Windows 10/11, macOS 10.15 Catalina и новее, а также дистрибутивы Linux с поддержкой .NET 8. Обязательным компонентом является среда исполнения .NET 8.

Условия развертывания:

- Приложение поддерживает автономную работу без постоянного подключения к интернету и предусматривает простое резервное копирование через копирование файла базы данных. Для сетевой версии требуется общий сетевой ресурс с правами чтения/записи, при этом установка может осуществляться самостоятельно без привлечения IT-специалистов.

					09.02.07.000022 П. 509	Лист
						21
Изм.	Лист	№ докум.	Подпись	Дата		

2.4 Обоснование выбора инструментальных средств разработки

Язык программирования

Разработка программного продукта требует осознанного выбора инструментальных средств, так как от этого напрямую зависят функциональные возможности и надёжность системы. В связи с этим был проведён анализ возможных технологий и инструментов разработки. При выборе учитывались такие критерии, как стабильность работы, бесплатность, совместимость компонентов между собой, а также удобство разработки и тестирования приложения.

В соответствии с требованиями юридической деятельности, система должна обеспечивать централизованный учёт документов, контрагентов и сроков с доступом для авторизованных сотрудников. Для реализации этих задач был выбран подход настольного приложения с использованием кроссплатформенного фреймворка.

На этапе проектирования рассматривались два основных варианта языка разработки – C# (с использованием Avalonia UI) и Java (с использованием JavaFX).

C# – это современный объектно-ориентированный язык программирования, разработанный компанией Microsoft. Он отличается высокой производительностью, безопасностью и строгой типизацией. C# широко используется для разработки настольных, мобильных и веб-приложений.

Avalonia UI – это кроссплатформенный фреймворк для создания настольных приложений, поддерживающий Windows, Linux и macOS. Он обеспечивает современный внешний вид, высокую производительность и использует XAML для описания пользовательского интерфейса.

Достоинства C# и Avalonia UI:

- высокая производительность и оптимизация;
- строгая типизация и безопасность кода;
- поддержка паттерна MVVM (Model-View-ViewModel);
- кроссплатформенность без изменения кода;
- хорошая интеграция с различными СУБД;
- богатая экосистема библиотек через NuGet.

Недостатки C# и Avalonia UI:

- необходимость установки .NET Runtime;
- ограниченная поддержка мобильных платформ;
- меньшее сообщество по сравнению с веб-технологиями.

Java – кроссплатформенный объектно-ориентированный язык программирования, отличающийся переносимостью и широким распространением в корпоративной среде.

JavaFX – платформа для создания настольных приложений на Java с современным пользовательским интерфейсом.

Достоинства Java и JavaFX:

- кроссплатформенность "write once, run anywhere";

- большое сообщество и обширные библиотеки;
- зрелая экосистема и инструменты разработки.

Недостатки Java и JavaFX:

- более высокое потребление памяти;
- менее современный подход к UI по сравнению с Avalonia.

Сложность настройки проектов.

После сравнения языков C# и Java предпочтение было отдано C# с использованием Avalonia UI, так как данная платформа обеспечивает более современный подход к разработке пользовательского интерфейса, лучшую производительность и удобство разработки.

Для взаимодействия с базой данных используется Entity Framework Core (EF Core) – современный ORM-фреймворк, который позволяет работать с данными через объектные модели без прямого написания SQL-запросов. Он поддерживает автоматическую генерацию миграций, упрощает обновление структуры базы данных и сокращает вероятность ошибок при работе с записями.

Система управления базами данных

Для хранения данных рассматривались три наиболее распространённые СУБД: Microsoft SQL Server, PostgreSQL и SQLite.

Microsoft SQL Server – коммерческая СУБД от Microsoft, предназначенная для корпоративных решений. Она обладает высокой производительностью и надёжной системой безопасности, но требует покупки лицензии.

Достоинства Microsoft SQL Server:

- высокая производительность и интеграция с .NET;
- поддержка сложных транзакций и хранимых процедур;
- надёжность и удобство резервного копирования.

Недостатки Microsoft SQL Server:

- необходимость приобретения лицензии;
- высокая нагрузка на систему при ограниченных ресурсах.

PostgreSQL – это мощная объектно-реляционная система управления базами данных с открытым исходным кодом. Она поддерживает сложные запросы, триггеры, процедуры и транзакции.

Преимущества PostgreSQL:

- полностью бесплатная и кроссплатформенная;
- поддержка сложных запросов и транзакций;
- высокая надёжность и отказоустойчивость.

Недостатки PostgreSQL:

- Требуется установки и настройки сервера;
- Более сложное администрирование для небольших проектов.

SQLite – встраиваемая реляционная система управления базами данных, не требующая отдельного серверного процесса. Данные хранятся в одном файле на диске.

Достоинства SQLite:

- не требует установки и настройки сервера;
- данные хранятся в одном файле, удобно для распространения;

- низкие требования к ресурсам;
- простота резервного копирования.

Недостатки SQLite:

- ограниченная производительность при большом количестве одновременных записей;
- отсутствие сетевого доступа к базе данных.

С учётом требований проекта и необходимости простоты развёртывания была выбрана SQLite. Она обеспечивает достаточную производительность для работы одного пользователя, простоту распространения приложения и не требует дополнительной установки программного обеспечения.

Среда разработки

Разработка приложения осуществляется в интегрированной среде Microsoft Visual Studio 2022, которая предоставляет разработчику все необходимые инструменты для написания, тестирования и отладки программного кода.

Visual Studio обладает следующими преимуществами:

- мощная система отладки и тестирования;
- поддержка системы контроля версий Git;
- встроенный менеджер пакетов NuGet для подключения библиотек;
- готовые шаблоны проектов Avalonia;
- удобный редактор XAML с предварительным просмотром;
- высокая производительность работы.

Дополнительные библиотеки и инструменты

Для реализации специфического функционала были выбраны следующие библиотеки:

- CommunityToolkit.Mvvm – реализация паттерна MVVM с кодогенерацией, упрощающая разработку привязок данных и команд;
- QuestPDF – библиотека для генерации PDF-документов с декларативным подходом к описанию макетов;
- DocX – библиотека для работы с документами формата Microsoft Word, позволяющая создавать и редактировать DOCX файлы;
- Newtonsoft.Json – популярная библиотека для работы с JSON форматом, используемая для сериализации и десериализации данных.

Архитектура приложения

Для обеспечения масштабируемости и поддерживаемости кода была выбрана архитектура MVVM (Model-View-ViewModel):

- Model – классы, отражающие предметную область и структуру базы данных;
- View – пользовательский интерфейс, реализованный на XAML;
- ViewModel – посредник между View и Model, содержащий логику представления.

Такой подход обеспечивает разделение ответственности, упрощает тестирование и позволяет независимо изменять интерфейс и бизнес-логику.

Система контроля версий

					09.02.07.000022 П. 509	Лист
						24
Изм.	Лист	№ докум.	Подпись	Дата		

Для контроля версий исходного кода используется Git с размещением репозитория на платформе GitHub. Это позволяет отслеживать изменения, сотрудничать с другими разработчиками и создавать резервные копии кода.

Проектирование пользовательского интерфейса

На этапе проектирования пользовательского интерфейса применялся современный подход с использованием онлайн-сервиса Figma - мощного инструмента для создания интерактивных прототипов и дизайн-систем.

Тестирование программного обеспечения

Для обеспечения надежности и корректности работы системы предусмотрено комплексное тестирование на различных уровнях разработки. Основное внимание уделяется модульному тестированию (unit testing) как фундаментальному методу проверки качества кода.

Юнит-тестирование реализуется с использованием фреймворка xUnit - современного решения для .NET приложений, которое отличается простотой использования и широкими возможностями настройки тестовых сценариев. Тестирование охватывает ключевые модули системы:

- тестирование сервиса аутентификации - проверка корректности верификации пользовательских данных, хеширования паролей и работы с солями;

- тестирование бизнес-логики - валидация операций с контрагентами, документами и шаблонами;

- тестирование валидации данных - проверка корректности вводимой информации, включая ИНН контрагентов, форматы дат и обязательные поля;

- тестирование генерации документов - верификация процесса создания PDF и DOCX файлов, замены плейсхолдеров в шаблонах.

Для изоляции тестируемых модулей используется библиотека Moq, позволяющая создавать mock-объекты зависимостей. Это обеспечивает независимость тестов от внешних систем, таких как база данных или файловая система.

Вывод по выбору инструментов

В ходе проведённого анализа различных инструментальных средств, применяемых при разработке программных продуктов, было принято решение использовать язык C# вместе с фреймворком Avalonia UI, систему управления базами данных SQLite, интегрированную среду разработки Microsoft Visual Studio, а также дополнительные библиотеки для работы с документами и PDF. Такой выбор обеспечивает высокую производительность, простоту развёртывания, удобство разработки и сопровождения, а также соответствует требованиям автоматизации юридической деятельности.

2.5 Разработка макетов основных форм приложения

На этапе проектирования интерфейса приложения были разработаны макеты основных окон, обеспечивающих удобный доступ к функционалу системы. Создание макетов определило будущую структуру основных форм, расположение элементов управления, а также логику взаимодействия пользователя с системой.

Основные разработанные экраны приложения:

- окно аутентификации - точка входа в систему, содержащая:
- поля для ввода логина и пароля;
- кнопку входа в систему;
- область для отображения сообщений об ошибках;
- информационную панель с подсказками.

Главное окно приложения - центральный интерфейс системы, включающий:

- боковую навигационную панель с основными разделами;
- область основного контента для работы с данными;
- панель статуса с информацией о текущем пользователе;
- кнопки управления приложением (выход, настройки).

Окно управления контрагентами - интерфейс для работы с клиентами и партнерами:

- табличное представление списка контрагентов;
- панель поиска и фильтрации;
- кнопки для добавления, редактирования и удаления записей;
- формы ввода и редактирования данных контрагентов.

Окно работы с документами - интерфейс для управления юридическими документами:

- список документов с возможностью сортировки;
- фильтры по типу, дате и статусу документов;
- редактор для создания документов;
- панель предпросмотра документов.

Макет окна входа для пользователя представлен на рисунке 2.1.

Макет главного окна приложения представлен на рисунке 2.2.

Макет окна для формирования отчетов и их поиска представлен на рисунке 2.3.

Макет окна для отслеживания процессуальных сроков представлен на рисунке 2.4.

Макет окна проверки контрагентов представлен на рисунке 2.5.

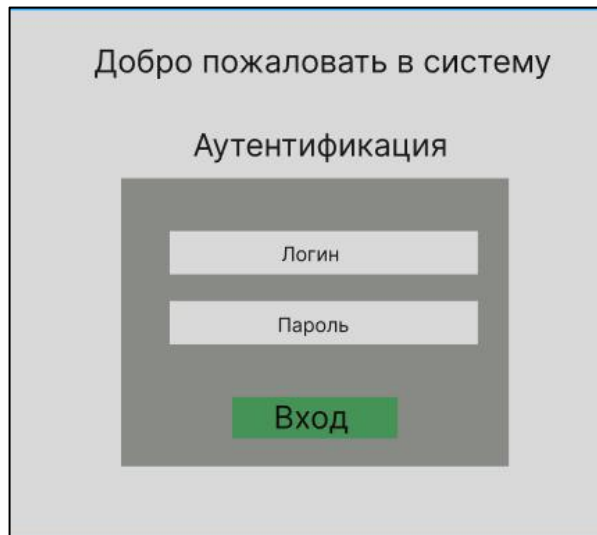


Рисунок 2.1 - Макет интерфейса окна входа

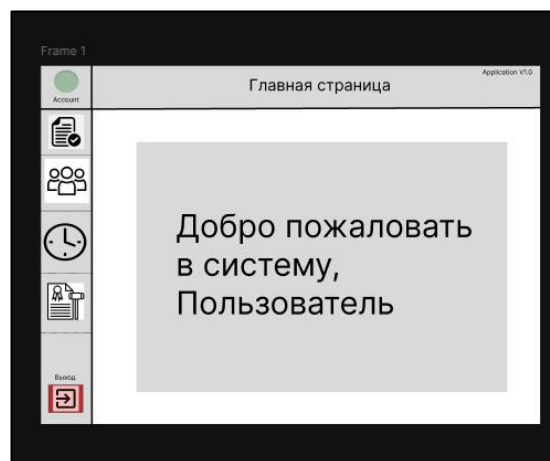


Рисунок 2.2 - Макет главного окна приложения

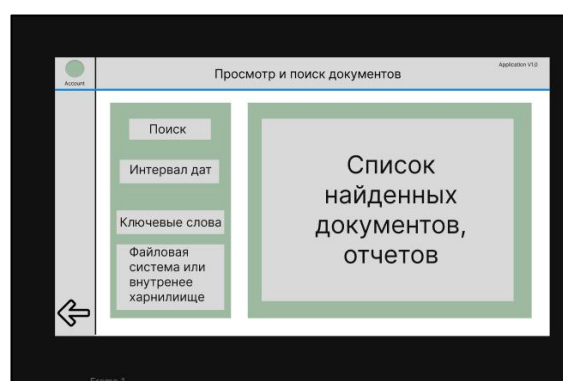


Рисунок 2.3 - Макет окна для формирования отчетов и их поиска

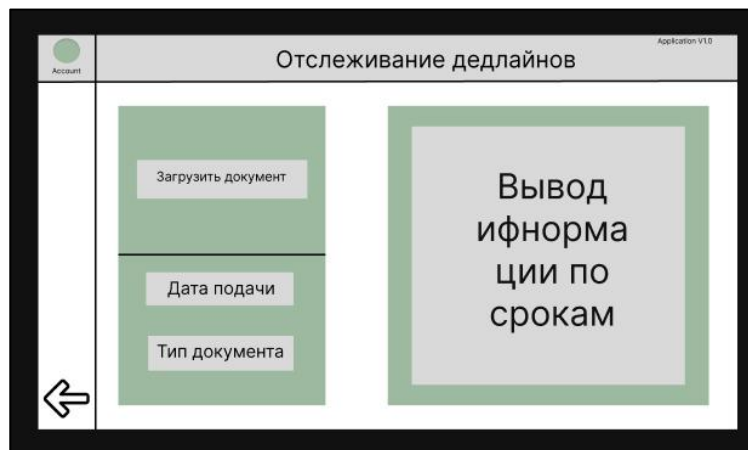


Рисунок 2.4 - Макет окна для отслеживания процессуальных сроков

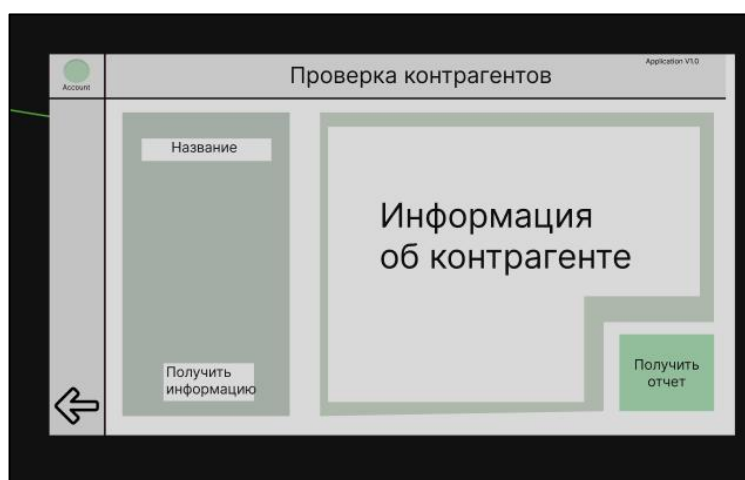


Рисунок 2.5 - Макет окна проверки контрагентов

2.6 Проектирование и разработка базы данных

Проектирование базы данных является одним из важнейших этапов разработки приложения, поскольку вся логика и интерфейс приложения основываются на структуре данных. На данном этапе была сформирована схема базы данных, а также была разработана сама база данных.

Структура базы данных, состоящая из сущностей и их атрибутов:

Пользователи (Users):

- id пользователя (первичный ключ);
- логин (уникальный);
- email;
- passwordHash (хеш пароля);
- salt (соль для шифрования);
- firstName (имя);
- lastName (фамилия);

- isActive (признак активности учетной записи);
- createdAt (дата создания).

Роли (Roles):

- Id роли (первичный ключ);
- Name (наименование роли: Администратор, Юрист, Наблюдатель);
- Description (описание роли);
- Permissions (права доступа).

Связь пользователей и ролей (UserRoles):

- userId (внешний ключ к Users);
- roleId (внешний ключ к Roles);
- составной первичный ключ (UserId, RoleId).

Контрагенты (Counterparties):

- id контрагента (первичный ключ);
- name (наименование организации);
- INN (ИНН, уникальный);
- address (юридический адрес);
- contactInfo (контактная информация);
- email;
- phone (телефон);
- createdAt (дата добавления).

Документы (Documents):

- id документа (первичный ключ);
- title (наименование документа);
- documentNumber (номер документа);
- content (содержание документа);
- createDate (дата создания);
- counterpartyId (внешний ключ к контрагенту);
- templateId (внешний ключ к шаблону).

Шаблоны документов (DocumentTemplates):

- id шаблона (первичный ключ);
- name (наименование шаблона);
- description (описание);
- templatePath (путь к файлу шаблона);
- isActive (признак активности).

Связи между сущностями в базе данных обеспечивают корректное отражение рабочих процессов юридической компании:

- связь «Пользователь – Роли» реализована через таблицу UserRoles (многие-ко-многим);
- связь «Контрагент – Документы» является отношением один-ко-многим;
- связь «Шаблон – Документы» обеспечивает наследование структуры документов.

Каждый документ связан с конкретным контрагентом и шаблоном

ERD - диаграмма базы данных представлена на рисунке 2.6.

					09.02.07.000022 П. 509	Лист
						29
Изм.	Лист	№ докум.	Подпись	Дата		

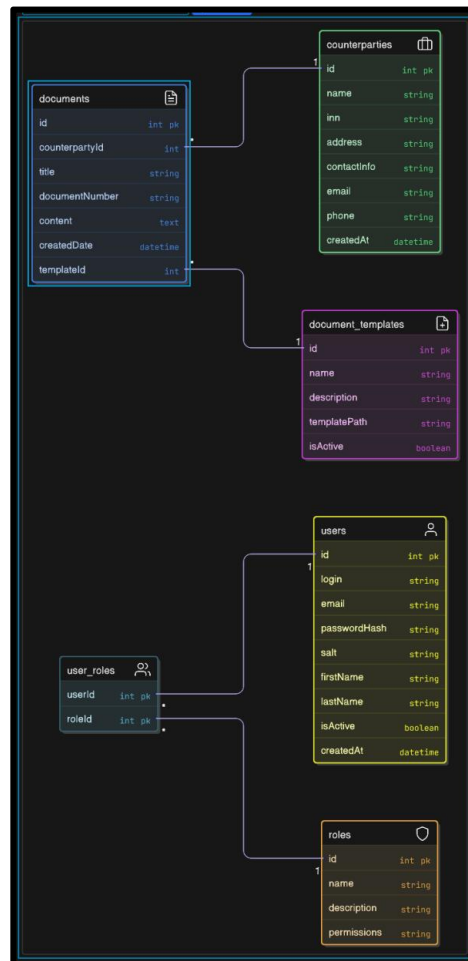


Рисунок 2.6 – ERD-диаграмма БД

2.7 Разработка пользовательского интерфейса

Разработка пользовательского интерфейса является важным этапом создания приложения, так как от удобства и понятности интерфейса зависит эффективность работы сотрудников юридической компании. На предыдущем этапе были сформированы макеты основных окон в сервисе Figma, которые определили структуру экранов, расположение элементов и общую логику взаимодействия пользователя с системой.

Процесс реализации пользовательского интерфейса включал следующие этапы:

- создание XAML-разметки для каждого окна приложения с использованием Avalonia UI;
- определение стилей и тем оформления для обеспечения единообразия интерфейса;
- реализация привязок данных между View и ViewModel с помощью Binding;
- настройка команд и обработчиков событий для пользовательских действий;

- оптимизация отзывчивости интерфейса и обработки состояний загрузки.

Ключевые элементы пользовательского интерфейса, реализованные в приложении:

- навигационная панель с иконками и текстовыми labels для основных разделов;

- таблицы данных с поддержкой сортировки, фильтрации и пагинации;

- формы ввода с валидацией данных и подсказками;

- модальные окна для подтверждения действий и дополнительных операций;

- система уведомлений для информирования пользователя о результатах операций;

- адаптивный интерфейс, обеспечивающий удобство работы на разных разрешениях экрана.

Цветовая схема и визуальное оформление:

- основная цветовая палитра - серо-зеленые тона (#2e7d32, #4caf50);

- фоновые цвета - светлые оттенки (#f8f9fa, #ffffff);

- акцентные цвета для действий - красный для опасных операций (#c62828);

- типографика - четкая иерархия шрифтов для удобочитаемости.

Реализованное окно входа пользователя представлено на рисунке 2.8.

Реализованное главное окно представлено на рисунке 2.9.

Реализованное окно «контрагенты» представлено на рисунке 2.10.

Реализованное окно «документы» представлено на рисунке 2.11.

Реализованное окно «события» представлено на рисунке 2.12.

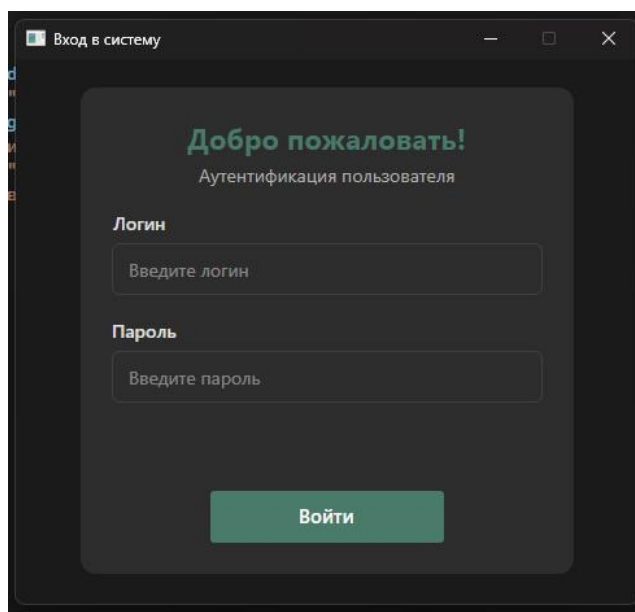


Рисунок 2.8 - Реализованное окно входа

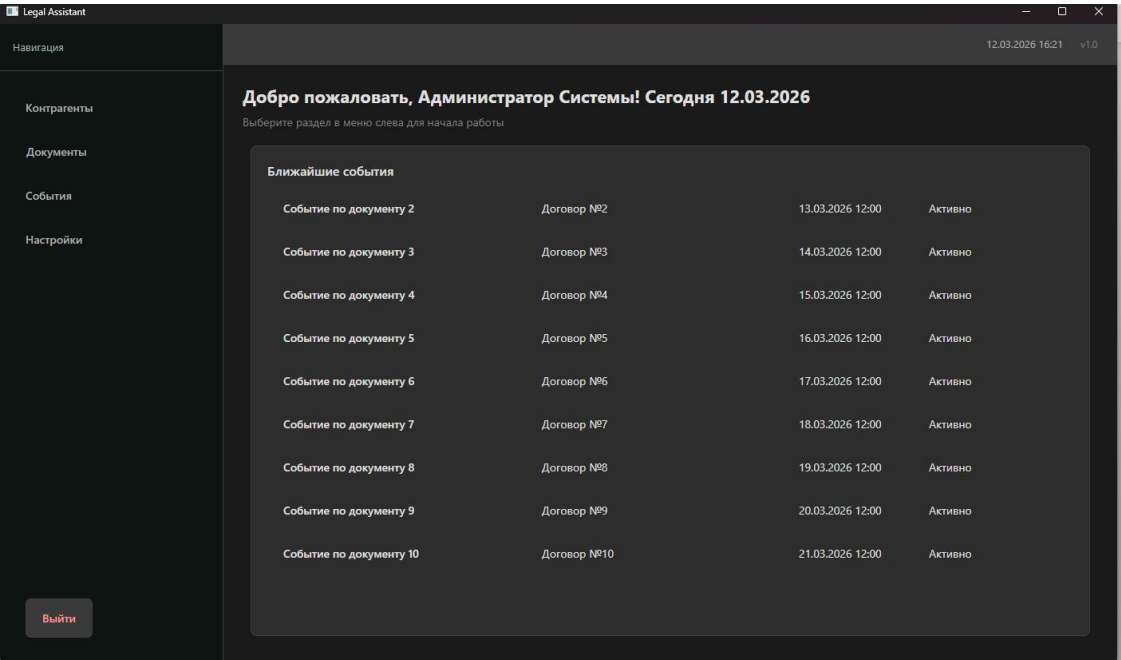


Рисунок 2.9 - Реализованное главное окно

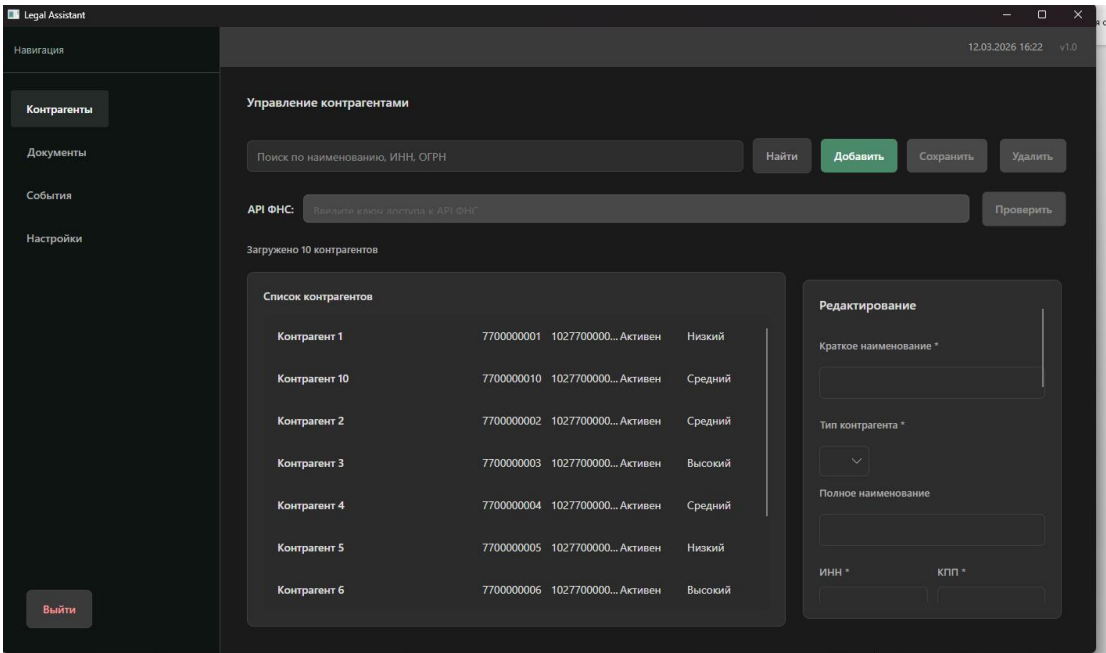


Рисунок 2.10 - Реализованное окно “контрагенты»

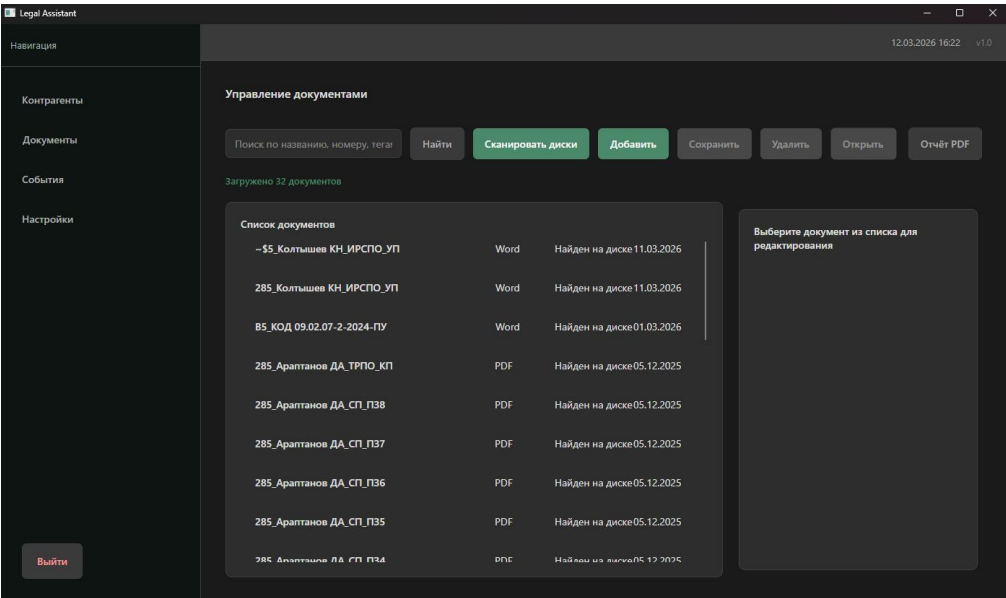


Рисунок 2.11 - Реализованное окно «документы»

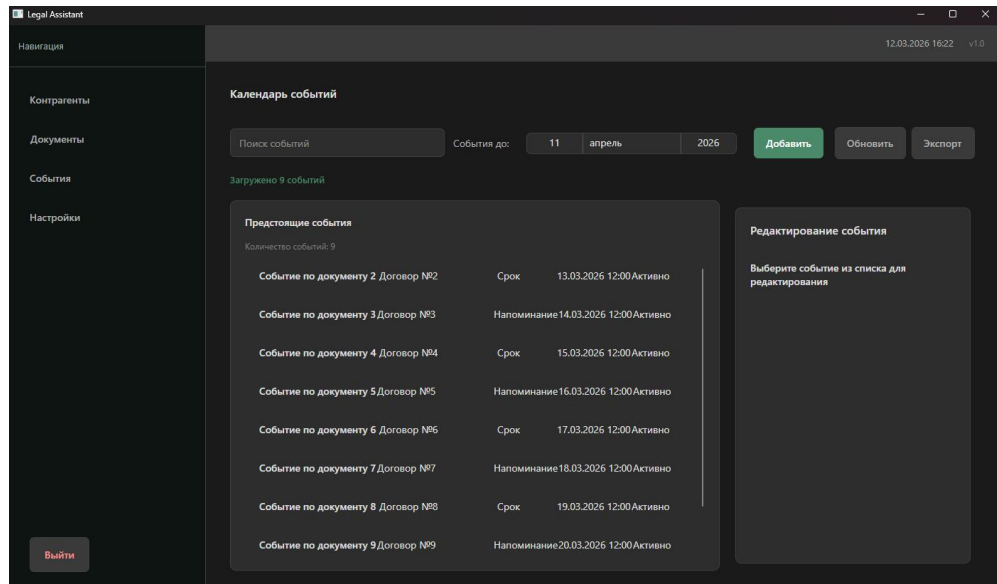


Рисунок 2.12 - Реализованное окно “события»

2.8 Доработка приложения

2.9.1 Определение доработок

Подтянуть состояние UI (Пользовательский интерфейс):
- сделать видимыми границы у всех полей ввода данных.

- на главном экране настроить вывод дат с цветовой пометкой: важные, ближайшие, просроченные.

- добавить и настроить переключение между светлой и темной темой.

Доработать логику алгоритмов:

- доделать систему напоминаний: настроить временные интервалы (за 5 мин, час, день, неделю).

- настроить приоритеты для событий на главном экране.

- реализовать логику: если дата просрочена — красный цвет, если сегодня/завтра — желтый.

Оптимизировать загрузку данных:

- ускорить загрузку списков контрагентов.

- ускорить открытие документов.

- сделать плавную загрузку данных на главном экране (без зависаний).

Доработать базовые функции:

- полностью доделать раздел "Контрагенты" (добавление, редактирование, удаление, поиск).

- полностью доделать систему напоминаний (чтобы они точно срабатывали и не терялись).

Добавить новую функцию (подключение к почте):

- реализовать подключение к email через SMTP.

- настроить автоматическую отправку напоминаний с документами на почту.

Сделать безопасный вход:

- проверить и доработать систему авторизации.

- убедиться, что пароли хранятся надежно.

- сделать защиту от несанкционированного доступа.

2.9.2 Разработка функционала

В ходе доработки приложения были реализованы следующие изменения.

Улучшение пользовательского интерфейса

Для всех полей ввода (TextBox) настроены единые стили: видимая рамка, четкий контур при фокусе и наведении, корректный цвет текста и плейсхолдера в светлой и тёмной теме. Теперь пользователь всегда понимает, где можно вводить данные и видит свой ввод.

На главном экране реализован блок «Ближайшие события», в котором события выводятся списком с датой и статусом. Логика формирования списка учитывает предстоящие даты, а сама разметка позволяет в дальнейшем выделять просроченные, сегодняшние и завтрашние события цветом.

Добавлено переключение между светлой и тёмной темой. Цветовые схемы (фон окна, карточек, текста, границ, состояний наведения и выбора) описаны

централизованно в ThemeManager, применяется нужная тема ко всем основным контролам, а выбранная тема сохраняется в пользовательских настройках.

Доработка логики уведомлений и событий

Модель события расширена полями ReminderOffsetMinutes, ReminderDate, ReminderEmailSent, ReminderLastSentDate, Priority. Это позволило реализовать гибкую систему напоминаний.

Для напоминаний добавлены стандартные интервалы: «за 5 минут», «за час», «за день», «за неделю». Выбор интервала в UI приводит к перерасчету ReminderDate относительно EventDate.

Сервис событий (EventService) реализует:

выборку предстоящих событий (с фильтром по дате и статусу),

выборку событий, по которым пора сработать напоминанию (по ReminderDate),

корректный пересчет даты напоминания при создании/редактировании.

Главный экран (ViewModel главного окна) регулярно (раз в 5 минут) опрашивает события на предмет напоминаний и обновляет счетчик напоминаний.

Оптимизация загрузки данных и отзывчивость интерфейса

Загрузка списков документов, событий и контрагентов переведена на асинхронные методы (async/await). Данные читаются из базы в фоновой задаче, а коллекции в UI обновляются через Dispatcher.UIThread, что исключает «подвисания» окна при больших выборках.

Для главного экрана реализован отдельный метод загрузки ближайших событий за период (по умолчанию 30 дней); при отсутствии данных дополнительно подгружаются все будущие события. Это обеспечивает быстрый и предсказуемый вывод информации при старте приложения.

Полноценный CRUD и поиск по контрагентам

В разделе «Контрагенты» реализован полный набор операций:

добавление нового контрагента (с инициализацией базовых полей и выбором его в форме),

редактирование выбранного контрагента,

сохранение (создание/обновление в базе),

удаление,

поиск по наименованию, ИНН и другим полям.

Вся логика вынесена в CounterpartyViewModel и CounterpartyService, работающих поверх AppDbContext (EF Core + SQLite). Статусы и уровни риска контрагентов приведены к человекочитаемому русскому виду.

Интеграция с почтой (SMTP) и автоматические уведомления

В AppSettings и на вкладке «Настройки» добавлен раздел SMTP: хост, порт, SSL, логин, пароль, адрес отправителя. Настройки сохраняются в JSON в папке AppData.

Создан EmailService, который:

формирует письма-напоминания по событиям,

при наличии пути к файлу документа прикрепляет его как вложение,

использует SmtClient с параметрами из настроек,

					09.02.07.000022 П. 509	Лист
						35
Изм.	Лист	№ докум.	Подпись	Дата		

логирует каждую отправку в таблицу EmailLogs (кому, когда, какой документ, статус, текст ошибки).

Главный ViewModel по таймеру получает список событий, требующих напоминаний, и для каждого вызывает EmailService. При успешной отправке в событии проставляются флаги ReminderEmailSent и время последней отправки.

Безопасная авторизация и хранение паролей

Сервис авторизации (AuthService) реализует:

аутентификацию по логину и паролю с загрузкой роли пользователя,

хранение пароля в виде хэша с солью (PBKDF2, SHA-256, 100k итераций),

проверку пароля с использованием защищённого сравнения (FixedTimeEquals).

В модели пользователя предусмотрены поля для аудита и защиты: FailedLoginAttempts, LockoutEnd, LastLoginDate, IsActive, а также связь с ролью.

В случае нескольких неверных попыток вход блокируется учетная запись на заданный интервал (например, 15 минут), что снижает риск перебора пароля.

После успешного входа счётчики ошибок и блокировка сбрасываются, обновляется время последнего входа. На UI показывается только сообщение об ошибке логина/пароля без утечки технических деталей.

Такой набор доработок перевел приложение из прототипа в более «production-ready» состояние: интерфейс стал понятнее, данные грузятся без подвисаний, напоминания и почтовые уведомления работают автоматически, а вход в систему соответствует базовым требованиям к безопасности.

Заключение

В процессе прохождения учебной практики все поставленные цели и задачи были успешно достигнуты. Основным результатом проделанной работы стала полностью готовая программа и пакет технической документации для сдачи демонстрационного экзамена, что также послужило надежной практической базой для реализации модуля дипломного проекта.

Для достижения этого результата был пройден полный цикл разработки информационной системы:

- спроектирована структура базы данных и написан функциональный программный код;
- настроена система безопасности с разграничением прав доступа пользователей по ролям;
- разработан удобный, интуитивно понятный и защищенный пользовательский интерфейс;
- составлена и оформлена вся необходимая проектная документация.

Все этапы создания проекта последовательно фиксировались в системе контроля версий Git. Итоговый программный продукт функционирует стабильно, без сбоев и эффективно решает реальные бизнес-задачи организации.

Библиография

- 1 Официальная документация по фреймворку Flask и языку Python [Электронный ресурс] – Режим доступа: <https://flask.palletsprojects.com/> ;
- 2 Документация по расширению Flask-SQLAlchemy для работы с базами данных [Электронный ресурс] – Режим доступа: <https://flask-sqlalchemy.palletsprojects.com/> ;
- 3 Официальная документация по Flask-Login для аутентификации пользователей [Электронный ресурс] – Режим доступа: <https://flask-login.readthedocs.io/> ;
- 4 Документация по языку шаблонов Jinja2 [Электронный ресурс] – Режим доступа: <https://jinja.palletsprojects.com/>;
- 5 Официальная документация СУБД SQLite [Электронный ресурс] – Режим доступа: <https://www.sqlite.org/docs.html>;
- 6 Интегрированная среда разработки PyCharm: официальное руководство [Электронный ресурс] – Режим доступа: <https://www.jetbrains.com/pycharm/documentation/>;
- 7 Официальная документация системы контроля версий Git [Электронный ресурс] – Режим доступа: <https://git-scm.com/doc>;
- 8 MDN Web Docs: Веб-технологии для разработчиков (HTML, CSS, JavaScript) [Электронный ресурс] – Режим доступа: <https://developer.mozilla.org/ru/>.

Приложение А
(справочная)
Листинг программы

Листинг импорта данных

```
import csv
import sys
import os
from datetime import datetime

sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from app import create_app, db
from app.models import User, Request

app = create_app()

def import_users():
    filename = os.path.join('data', 'import_users.csv')
    count = 0
    with open(filename, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f, delimiter=';')
        for row in reader:
            user = User(
                id=int(row['userID']),
                fio=row['fio'],
                phone=row['phone'],
                login=row['login'],
                password=row['password'],
                type=row['type']
            )
            db.session.add(user)
            count += 1
        db.session.commit()
    return count

def import_requests():
    filename = os.path.join('data', 'import_requests.csv')
    count = 0
    with open(filename, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f, delimiter=';')
        for row in reader:
            number = f"REQ-{int(row['requestID']):04d}"
            start_date = datetime.strptime(row['startDate'],
            '%Y-%m-%d').date()
            completion_date = None
            if row['completionDate'] and
            row['completionDate'].lower() != 'null':
```

					09.02.07.000022 П. 509	Лист
						39
Изм.	Лист	№ докум.	Подпись	Дата		

```

        completion_date =
datetime.strptime(row['completionDate'], '%Y-%m-%d').date()
        master_id = int(row['masterID']) if row['masterID']
and row['masterID'].lower() != 'null' else None
        request = Request(
            request_id=int(row['requestID']),
            number=number,
            start_date=start_date,
            climate_tech_type=row['climateTechType'],
            climate_tech_model=row['climateTechModel'],
            problem_description=row['problemDescription'],
            request_status=row['requestStatus'],
            completion_date=completion_date,
            repair_parts=row['repairParts'] if
row['repairParts'] else None,
            master_id=master_id,
            client_id=int(row['clientID'])
        )
        db.session.add(request)
        count += 1
        db.session.commit()
    return count

if __name__ == '__main__':
    with app.app_context():
        db.drop_all()
        db.create_all()
        users = import_users()
        requests = import_requests()
        print(f"Imported: {users} users, {requests} requests")

```


Приложение Б
(справочная)
Листинг программы

Файл debug_db.py

```
import os
from sqlalchemy import inspect, create_engine

db_files = []
for root, dirs, files in os.walk('.'):
    for file in files:
        if file.endswith('.db'):
            path = os.path.join(root, file)
            size = os.path.getsize(path)
            db_files.append((path, size))

print(f"Found {len(db_files)} database files:")
for path, size in db_files:
    print(f"    {path} ({size} bytes)")
    try:
        engine = create_engine(f'sqlite:/// {path}')
        inspector = inspect(engine)
        tables = inspector.get_table_names()
        print(f"        Tables: {tables}")
    except Exception as e:
        print(f"        Error: {e}")

try:
    from app import create_app, db

    app = create_app()
    with app.app_context():
        db_path =
app.config['SQLALCHEMY_DATABASE_URI'].replace('sqlite:///', '')
        print(f"\nApp database: {os.path.abspath(db_path)}")

        if os.path.exists(db_path):
            print(f"    File exists, size: {os.path.getsize(db_path)}
bytes")

            inspector = inspect(db.engine)
            tables = inspector.get_table_names()
            print(f"    Tables: {tables}")
except Exception as e:
    print(f"Error checking app: {e}")
```

Файл __init__.py

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager
import os

db = SQLAlchemy()
login_manager = LoginManager()

def create_app():
    app = Flask(__name__)
    app.config['SECRET_KEY'] = 'your-secret-key-change-this'

    basedir =
os.path.abspath(os.path.dirname(os.path.dirname(__file__)))
    db_path = os.path.join(basedir, 'instance', 'repair_system.db')
    app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:/// ' + db_path
    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

    instance_path = os.path.join(basedir, 'instance')
    if not os.path.exists(instance_path):
        os.makedirs(instance_path)

    db.init_app(app)
    login_manager.init_app(app)
    login_manager.login_view = 'login'

    from app import models

    with app.app_context():
        from app.routes import init_routes
        init_routes(app)

    return app
```

					09.02.07.000022 П. 509	Лист
						42
Изм.	Лист	№ докум.	Подпись	Дата		